Mongoose

Mongoose is an elegant MongoDB object modeling library for Node.js that helps you structure data with schemas, enforce validation, add middleware/hooks, and build expressive queries.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});

const User = mongoose.model('User', userSchema);
```

mongoosejs.com

Mongoose ODM v9.4.1

Let's face it, writing MongoDB validation, casting and business logic boilerplate is a drag. That's why we wrote Mongoose.

Schema

Schema defines the shape of the documents within that collection.

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number,
});
```

Models

Model in mongoose is a class with which we construct documents.

```
const User = mongoose.model("User", userSchema);
```

Automatic Collection Naming in MongoDB

When you define a model with `mongoose.model('User', userSchema)`, Mongoose automatically converts the model name into the MongoDB collection name by:

- Lowercasing the model name
- Pluralizing it
- Using the result as the collection name

Examples:

```
const User = mongoose.model('User', userSchema);      // → collection: 'users'
const Person = mongoose.model('Person', personSchema); // → collection: 'people'
const Category = mongoose.model('Category', catSchema); // → collection: 'categories'
```

You can override the default collection name either by passing the third argument to `mongoose.model()` or by setting `collection` in the schema options:

INSERT

Inserting One

```
const user1 = new User({ name: "Adam", email: "adam@yahoo.in", age: 48 });
const user2 = new User({ name: "Eve", email: "eve@google.com", age: 48 });


user1.save()  //to save in DB

user2.save()  //to save in DB
```

example: as it returns a promise user1.save().then()

INSERT

Inserting Multiple

```
User.insertMany([
  { name: "Tony", email: "tony@gmail.com", age: 50 },
  { name: "Bruce", email: "bruce@gmail.com", age: 47 },
  { name: "Peter", email: "peter@gmail.com", age: 30 },
]).then((data) => {
  console.log(data);
});
```

FIND

Model.find( ) //returns a Query Object (thennable)

***Mongoose Queries are not promises. But they have a .then( )**

```
User.find().then((data) => {
  console.log(data);
});
```

```
User.find({ age: { $gte: 47 } }).then((data) => {
  console.log(data);
});
```

```
User.find({ age: { $gt: 48 } })
  .then((res) => {
    console.log(res[0].name);
  })
  .catch((err) => {
    console.log(err);
  });
```

UPDATE

Model.updateOne()

```
User.updateOne({ name: "Bruce" }, { age: 49 }).then((res) => {
  console.log(res);
});
```

Model.updateMany()

```
User.updateMany({ age: { $gt: 45 } }, { age: 45 }).then((res) =
  console.log(res);
});
```

Model.findOneAndUpdate( )

```
User.updateMany({ age: { $gt: 45 } }, { age: 45 }).then((res) => {
  console.log(res);
});
```

```
User.findOneAndUpdate({ name: "Tony" }, { age: 60 }, { new: true }).then(
  (data) => {
    console.log(data);
  }
);
```

DELETE

Model.deleteOne() *//returns count*

```
User.deleteOne({ name: "Adam" }).then((res) => {
  console.log(res);
});
```

Model.deleteMany() *//returns count*

```
User.deleteMany({ age: { $gt: 40 } }).then((res) => {
  console.log(res);
});
```

Model.findByIdAndDelete()

Model.findOneAndDelete()

Schema Validations

Basically, Rules for Schema

```
const bookSchema = mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  author: {
    type: String,
  },
  price: {
    type: Number,
  },
});
```

```
const Book = mongoose.model("Book", bookSchema);

let book1 = new Book({
    title: "Mathematics XII",
    author: "RD Sharma",
    price: 1200
});
book1.save().then(res => {
    console.log(res);
})
```

SchemaType Options

```
const bookSchema = mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  author: {
    type: String,
  },
  price: {
    type: Number,
    min: [1, "please enter a valid price"],
  },
  discount: {
    type: Number,
    default: 0,
  },
  genre: [String],
  category: {
    type: String,
    enum: ["fiction", "non-fiction"],
```

```
Book.findByIdAndUpdate("6501a3a06efc64235a16333c", { price: -500 })
  .then((res) => {
    console.log(res);
  })
  .catch((err) => {
    console.log(err);
  });
```

```
{runValidators: true}
```

Display custom error message

```
price: {
  type: Number,
  min: [1, "Price is too low for Amazon selling"],
},
```

```
console.log(err.errors.category.properties.message);
```

```
.catch((err) => {
  console.log(err.errors.price.properties.message);
});
```